Philosophy of the GSP Core Package

1. Preamble

The `gsp` package is the **contract layer** of GSP_API: the abstract protocol that every backend implements and that every convenience layer builds on. It defines what a `Canvas`, `Viewport`, `Camera`, `Buffer`, and `Visual` *are*; it does not draw anything.

This document is a focused tour of `src/gsp/`. The two companion docs already cover the wider ecosystem and the renderer side:

- philosophy_packages.md — the seven-package split, with §3.1 a one-page sketch of `gsp` that this document expands.
- philosophy_renderers.md — the `RendererBase` contract and how the three backends consume it.

Audience. A contributor opening `src/gsp/` for the first time, or one who has read `philosophy_packages.md` and wants the deeper drill into the contract layer.

The two-sentence rule (reused from philosophy_packages.md §1) governs the split inside `gsp` too:

> *Each subpackage should be understandable alone, and the package should be understandable as a whole.*

One-line proof of independence. The whole package is numpy + stdlib only:

```
grep -rn "^from gsp_\|^import gsp_" src/gsp/   # no matches
```

If this command ever returns a hit, `gsp` has acquired a dependency on a sibling — and the layered architecture (philosophy_packages.md §2.4) is broken.

---

2. Design Philosophy

Five principles. Together they are the "why" before the "what" of every section that follows.

2.1 Contract, not implementation

`gsp` defines *what* a renderer must do, never *how*. The six-method `RendererBase` is a pure ABC; the three backend packages are the only direct subclasses. The same is true of `AnimatorBase`, `ViewportEventsBase`, `SerializerBase`, and `TransformLinkBase` — all abstract, all subclassed outside this package. Verification:

```
grep -rn "class .*RendererBase\|class .*AnimatorBase\|class .*ViewportEventsBase"
src/gsp/types/
# only the abstract bases themselves; no concrete implementations
```

2.2 Data, not commands

A `Visual` is a record of attributes — positions, colors, sizes — not a procedure. There is no `Visual.draw()`. Rendering is the verb owned by `RendererBase.render(viewports, visuals, model_matrices, cameras)` (renderer_base.py) — visuals appear there as *arguments*, never as actors. Same for `Canvas`, `Viewport`, `Camera`, `Texture`, `Geometry`: they are inert data containers.

This separation is what lets a single scene description run unchanged through every backend.

2.3 Lazy data via `TransBuf`

Every place a `Visual` or `Geometry` or `Camera` accepts buffer-shaped data, the type is not `Buffer` — it is `TransBuf = TransformChain | Buffer` (transbuf.py:11). A field can hold either a concrete `Buffer` *or* a `TransformChain` that produces one when run. The chain is executed at render time, by the backend, via `TransBufUtils.to_buffer`. This is the lazy-evaluation seam — see §6.

2.4 Self-registration over manifests

Two registries — `RendererRegistry` and `TransformRegistry` — let backends and transform-link types plug in by *importing* themselves. There is no central manifest of available renderers or links; the act of `import gsp_matplotlib` calls `RendererRegistry.register_renderer(...)` at module top level. This is the same pattern in both directions, and it is what makes `pip install gsp_<backend>` enough to make the backend visible.

2.5 Numpy + stdlib only

Already shown in §1. Stated once more because it is a structural property: the contract layer carries no library dependencies that consumers don't already have.

---

3. Architecture: the seven subpackages

The package is split along the same lines that the rest of the system is. Each subpackage is small enough to read in one sitting, and each has a single role.

Subpackage	Role	Imports from inside `gsp`
`gsp.types`	Abstract bases + value types — the lowest layer	(none — leaf)
`gsp.core`	Scene-graph containers (`Canvas`, `Viewport`, `Camera`, `Texture`, `Event`)	`types`, `utils`
`gsp.transforms`	`TransformChain`, `TransformLinkBase`, `TransformRegistry`, in-core links	`types`
`gsp.geometry`	`Geometry`, `MeshGeometry` — vertex containers	`types`
`gsp.materials`	`Material` and friends — material descriptors	`types`
`gsp.visuals`	The eight concrete visuals	`types`, `core`, `geometry`, `materials`
`gsp.utils`	Free functions and the registries	`types`, `core`

Plus two top-level files: `gsp/__init__.py` (six imports, no logic) and `gsp/constants.py` (`Constants.FaceCulling` and `Constants.Color` presets).

Internal dependency direction. `types` is the leaf — every other subpackage depends on it. `transforms`, `geometry`, `materials` build on `types` only. `visuals` builds on all of the above. `utils` and `core` cross-cut.

The deliberate non-re-export. `gsp/types/__init__.py:16–18` does *not* re-export `RendererBase` or `SerializerBase`:

```
# FIXME those 2 are creating a circular import
# from .renderer_base import RendererBase
# from .serializer_base import SerializerBase
```

The reason is that those two modules import from `core` (for `Canvas` / `Viewport`), and `core` itself depends on `types`. Users must import them from the submodule directly:

```
from gsp.types.renderer_base import RendererBase
from gsp.types.serializer_base import SerializerBase
```

Verification: `cat src/gsp/types/__init__.py | tail -5`.

---

4. Core Concept: Buffer

A `Buffer` is the smallest unit of typed bulk data the protocol knows about. It is the wire-format the contract layer speaks.

4.1 The class

`Buffer` is a typed, single-dimension array. From its docstring:

> *Typed array with single dimension. It is immutable in count and type, but mutable in content.*

Three fields, set once at construction and never resized (buffer.py:19-29):

```
self._count: int = count
self._type: BufferType = buffer_type
self._bytearray: bytearray = bytearray(count * item_size)
```

The element count and type are part of the buffer's identity; only the bytes inside are mutable. `Buffer` exposes `get_data(offset, count)` (which returns a *new* sliced `Buffer`), `set_data(bytes, offset, count)` (copy semantics), and `to_bytearray()` / `from_bytearray()` for serialization.

4.2 `BufferType` — the type axis

`BufferType` is a 12-variant enum, modelled on GLSL types:

Group	Variants
Scalars	`float32`, `uint32`, `uint8`, `int32`, `int8`
Vectors (float32)	`vec1`, `vec2`, `vec3`, `vec4`
Vector (uint32)	`uvec4`
Matrix	`mat4` (4×4 column-major float32)
Color	`rgba8` (4 unsigned bytes)

The enum carries three static helpers (buffer_type.py:38-132): `get_item_size(type)` returns bytes-per-element, `to_numpy_dtype(type)` and `from_numpy(arr)` bridge to numpy.

4.3 The numpy bridge

`Buffer` itself does *not* hold a numpy array — it holds raw bytes. The actual numpy adapter, `Bufferx`, lives in `gsp_extra` and is the helper users call in practice (`Bufferx.from_numpy(arr, BufferType.vec3)`). This split is deliberate: `Buffer` in the contract layer is byte-level so that serialization, network transport, and GPU-upload can all speak it natively; the numpy ergonomics live one layer up.

4.4 Where `Buffer` is consumed

Almost every data-bearing slot in the protocol is a `TransBuf`, which resolves to a `Buffer` at render time. The consumers:

- Every `Visual` — positions, colors, sizes, etc. (§5)
- `Camera` — `view_matrix` and `projection_matrix` (`mat4` buffers)
- `Texture` — pixel `data` (`rgba8` buffer)
- `Geometry` — `positions` (`vec3` buffer)

To verify: `grep -rn "TransBuf" src/gsp/visuals/ src/gsp/core/ src/gsp/geometry/` .

---

5. Core Concept: Visual

A `Visual` is a record of *what* you want drawn, not a *how*. Every concrete visual is a data container; the rendering happens elsewhere.

5.1 Evidence: data container, not draw command

- `VisualBase` has no `render()` method. Its only members are `_uuid` and `userData`:

```
class VisualBase:
    __slots__ = ["_uuid", "userData"]
    def __init__(self):
        self._uuid: str = UuidUtils.generate_uuid()
        self.userData: dict[str, Any] = {}
```

- The verb `render` lives on `RendererBase`; visuals appear there as **arguments**.
- Each backend's `_render_visual()` is a hand-written `isinstance` chain over the eight visual classes — see philosophy_renderers.md §5.1. The visual is the *subject* of dispatch, never the actor.

5.2 The catalogue

The package ships eight concrete visuals; this document covers seven (Mesh is documented separately):

Visual	Adds (every field below is `TransBuf` unless noted)	File
`Points`	`positions`, `sizes`, `face_colors`, `edge_colors`, `edge_widths`	points.py
`Markers`	`Points` fields + `marker_shape` (`MarkerShape` enum)	markers.py
`Pixels`	`positions`, `colors`, `groups` (`Groups` value type)	pixels.py

`Paths`	`positions`, `path_sizes`, `colors`, `line_widths`, `cap_style`, `join_style`	paths.py
`Segments`	`positions`, `line_widths`, `colors`, `cap_style`	segments.py
`Texts`	`positions`, `colors`, `font_sizes`, `angles`, plus `strings`, `textAligns`, `font_name` (Python lists / strings, not buffers)	texts.py
`Image`	`texture` (`Texture`), `position`, `image_extent`, `image_interpolation`	image.py

5.3 The universal pattern

Every visual follows the same shape:

- Per-attribute getter/setter pair.
- A `set_attributes(...)` method that takes optional kwargs and writes them in one call.
- A `check_attributes()` instance method that delegates to a static `sanity_check_attributes(...)` (and `sanity_check_attributes_buffer(...)` for the post-conversion check).
- `__slots__` on every visual — no surprise attributes.

Most validation bodies are stubs (`pass`); `Texts` is the most complete, asserting that `positions` , `colors` , `font_sizes` , `angles` , and `textAligns` all match the string count (texts.py:177-211).

To verify: `grep -rn "class.*VisualBase" src/gsp/visuals/` returns eight matches (the seven above plus `Mesh`).

---

6. Core Concept: Transform

A `TransformChain` is a CPU-side, lazy data-transformation pipeline. It is what makes a `Buffer` slot something you can populate without computing the bytes up front — and something you can serialize and re-execute on another machine.

6.1 The motivation in one type

`transbuf.py:11` :

```
TransBuf = TransformChain | Buffer
```

Every buffer-shaped field on every visual / camera / texture / geometry has type `TransBuf` . A consumer can hand in a fully-baked `Buffer` *or* a description of how to make one. The renderer doesn't care which until render time.

6.2 `TransformChain`

`TransformChain` holds three things (lines 20-40): an ordered list of `TransformLinkBase` instances, the expected output `buffer_count` , and the expected output `buffer_type` (either may be left undefined as `-1` / `None`).

The execution model is a pipe-fold (`run()` at lines 126-149):

```
def run(self) -> Buffer:
    buffer = None
```

```
    for link in self.__links:
        buffer = link.apply(buffer)
    assert buffer is not None
    return buffer
```

Each link reads the previous link's output and produces a new `Buffer`. The first link receives `None` — which is the convention for "source" links that load or generate data.

6.3 `TransformLinkBase`

`TransformLinkBase` is a three-method ABC:

```
class TransformLinkBase(ABC):
    @abstractmethod
    def apply(self, buffer_src: Buffer | None) -> Buffer: ...
    @abstractmethod
    def serialize(self) -> dict[str, Any]: ...
    @staticmethod
    @abstractmethod
    def deserialize(data: dict[str, Any]) -> "TransformLinkBase": ...
```

`apply` does the work; `serialize` / `deserialize` make the chain transportable.

6.4 `TransformRegistry`

`TransformRegistry` is the deserialization seam, mirroring `RendererRegistry`. It is a string→class map: a serialized link carries a `link_type` string, and on deserialization the chain looks up the class via `TransformRegistry.get_link_class(name)` and calls its `deserialize(data)`. Concrete link modules self-register at import time, the same way backends do.

6.5 Concrete links

Two by default — one in core, one in `gsp_extra`:

Link	Where	Role
`TransformLinkImmediate`	`gsp/transforms/links/transform_link_immediate.py`	Returns a fixed `Buffer`; ignores its input. The "constant" of the system.
`TransformLoad`	`gsp_extra/transform_links/transform_load.py`	Loads from a URI (file, `.npy`, image, HTTP) into a `Buffer`. Source link.

The split is deliberate: in-core links are minimal (no I/O, no decoders); URI loading is a `gsp_extra` concern.

6.6 Where the chain runs

Backends — not `gsp` itself — execute chains. Every per-visual renderer in `gsp_matplotlib` and `gsp_datoviz` calls `TransBufUtils.to_buffer(trans_buf)` before reading data:

```
if isinstance(trans_buf, Buffer):
    return trans_buf
elif isinstance(trans_buf, TransformChain):
    return trans_buf.run()
```

To verify: `grep -rn "TransBufUtils.to_buffer" src/` .

6.7 Why this matters

Because chains are serialisable, a scene with `TransformChain` slots can travel through `gsp_pydantic` to JSON, across the wire via `gsp_network` , and re-execute on the rendering host. The source data never has to be on the same machine as the data definition.

---

7. The other contracts

Four more abstract bases live in `gsp.types/` . Each is small; each is implemented exactly N times where N is the number of backends.

Contract	Methods (abstract)	Implemented by
`RendererBase`	`__init__(canvas)`, `render(...)`, `show()`, `close()`, `clear()`, `get_canvas()` — see philosophy_renderers.md §4	the three backends
`AnimatorBase`	`__init__(renderer)`, `add_callback`, `remove_callback`, `event_listener` (decorator), `start(viewports, visuals, ...)`, `stop()`. Plus public `on_video_saved: Event`	the three backends
`ViewportEventsBase`	`__init__(renderer, viewport)`. Plus seven public `Event` slots: `key_press_event`, `key_release_event`, `button_press_event`, `button_release_event`, `mouse_move_event`, `mouse_scroll_event`, `canvas_resize_event`	the three backends
`SerializerBase`	`serialize(viewports, visuals, model_matrices, cameras) -> dict`	`gsp_pydantic`

The animator callback shape is `AnimatorFunc: Callable[[float], Sequence[VisualBase]]` (animator_types.py) — given an elapsed time, return the visuals that changed this frame. Returning a short list is what lets the renderer's lazy-create / mutate-update cache stay efficient (philosophy_renderers.md §5.3).

The `Event` class itself is a small generic pub/sub primitive: `subscribe` , `unsubscribe` , `dispatch` , plus an `event_listener` decorator form. It is the only piece of behaviour-bearing code in `core/` ; everything else there is data.

---

8. The scene-graph containers (`gsp.core`)

Five classes. All of them are pure data — fields with getters and setters, no methods that draw.

Class	Holds	Notable

Canvas	width, height, dpi, background_color, userData	Root render surface. Backends size their figure / app from these.
Viewport	x, y, width, height, background_color, userData	Rectangular sub-region of the canvas. Pixel coords, origin bottom-left.
Camera	view_matrix (TransBuf), projection_matrix (TransBuf), userData	One of the four parallel lists RendererBase.render consumes.
Texture	data (TransBuf), width, height, userData	Held by Image visual.
Event	generic Callback type parameter	The pub/sub primitive used by ViewportEventsBase and AnimatorBase.

Every UUID-bearing container also exposes a `userData: dict[str, Any]` slot — the protocol's escape hatch for application-specific metadata.

---

9. Value types and enums

Small types that carry no buffer of their own. All live in `gsp.types/`.

Name	Kind	Used by
Color	TypeAlias = bytearray (4 bytes, RGBA)	every visual; presets in Constants.Color
Groups	TypeAlias = int | list[int] | list[list[int]]	Pixels for batching
MarkerShape	Enum (13 values: disc, square, triangle_*, cross, diamond, ...)	Markers
CapStyle	Enum (BUTT, PROJECTING, ROUND)	Paths, Segments
JoinStyle	Enum (MITER, BEVEL, ROUND)	Paths
TextAlign	IntEnum, value = vertical*10 + horizontal (9 values from TOP_LEFT(0) to BOTTOM_RIGHT(22))	Texts
ImageInterpolation	Enum (LINEAR, NEAREST)	Image

The IntEnum trick on `TextAlign` is worth flagging: `value // 10` recovers the vertical axis, `value % 10` the horizontal — backends can decompose alignment without a lookup table.

---

10. Utils and constants

`gsp.utils/` is a flat collection of static-class helpers. None of them carry state across calls.

Module	What it provides
renderer_registery.py	RendererRegistry — backend triad registration. Full coverage in philosophy_renderers.md §6.
transbuf_utils.py	TransBufUtils.to_buffer(trans_buf) -> Buffer — the §6.6 dispatch.

uuid_utils.py	`UuidUtils.generate_uuid()` — UUID v4, deterministic when `GSP_UUID_COUNTER` is set (test mode).
math_utils.py	`MathUtils.apply_mvp_to_vertices_transform(...)` — CPU-side MVP for backends that don't transform on the GPU.
cmap_utils.py	Colormap name lookup against matplotlib's registry.
group_utils.py	`GroupUtils.get_group_count(...)` — interprets the `Groups` value type.
unit_utils.py	Inch/cm and point/pixel conversions, DPI-aware.
viewport_unit_utils.py	Viewport pixel ↔ NDC conversion.
log_utils.py	A pre-configured loguru `logger`.

`gsp/constants.py` ships two things, both flat and useful:

- `Constants.FaceCulling` — Enum: `FrontSide(0)`, `BackSide(1)`, `BothSides(2)`.
- `Constants.Color` — named `bytearray` colour presets: `white`, `black`, `red`, `green`, `blue`, `yellow`, `magenta`, `cyan`, `light_gray`, `gray`, `dark_gray`, `transparent`.

---

11. Relations with other packages

`gsp` sits at the bottom of the dependency graph. The arrows all point inward.

Consumer	What it takes from `gsp`	What it adds
gsp_matplotlib	`RendererBase`, `AnimatorBase`, `ViewportEventsBase`, all visuals/types	matplotlib-backed renderer triad; multi-format export (PNG/SVG/PDF)
gsp_datoviz	same three contracts	GPU/interactive renderer triad
gsp_network	same three contracts	thin-client renderer triad over HTTP
gsp_pydantic	`SerializerBase`, the whole data tree	pydantic models + parser, JSON round-trip
gsp_extra	abstract types	`Object3D` scene graph, `Bufferx` numpy bridge, camera controls, `TransformLoad`
vispy2	abstract types (via `gsp_extra`)	matplotlib-like facade (`scatter`, `plot`, `imshow`, `Axes*`)

The contract surface a consumer touches is small. From each row above, the seam is named: a backend implements three abstract bases and registers them once; `gsp_pydantic` implements one; the convenience packages implement none and just consume.

The dependency table in philosophy_packages.md §2.4 shows the same picture from the outside: every other package has `gsp` as a dependency; nothing the other way.

To verify the inward-only flow:

```
grep -rn "^from gsp_\|^import gsp_" src/gsp/   # empty
```

12. Verification — read this document against the code

Every claim above is grounded in a specific file or grep recipe. The shell commands below confirm the structural claims directly.

1. **Independence (§1, §2.5, §11)** — `gsp` imports no sibling:

```
grep -rn "^from gsp_\|^import gsp_" src/gsp/
```

2. **Visuals all extend `VisualBase` (§5)** — eight matches (the seven documented plus `Mesh`):

```
grep -rn "VisualBase" src/gsp/visuals/
```

3. **Transform plumbing (§6)** — registry, base, in-core link:

```
grep -rn "TransformLinkBase" src/gsp/
```

4. **Backends are the ones that run chains (§6.6)**:

```
grep -rn "TransBufUtils.to_buffer" src/
```

5. **The non-re-export quirk (§3)** — read the bottom of:

```
cat src/gsp/types/__init__.py
```

6. **Five enums and two type aliases in the value layer (§9)**:

```
ls
src/gsp/types/{color,group,marker_shape,cap_style,join_style,text_align,image_ir
```

When the code drifts from this document — change the document. The `gsp` package is the source of truth; this file just names what is already in it.